

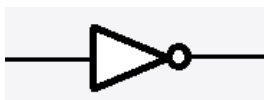
## 159.102 Notes

### Section 1 : Gates, Circuits and Boolean Algebra

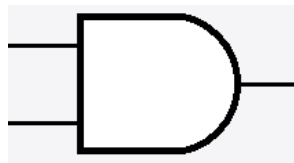
In Computer Science (the study of what is possible with computers), it is important to understand the machine that we work with. Computers are made up of thousands of electronic circuits. In this section we will describe the basic components of a circuit and how to design circuits for specific purposes.

#### Gates

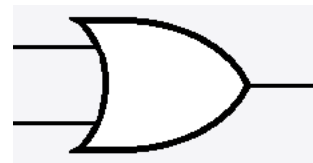
The basic components of a computer are **gates**. These are real components that you can buy and use to make electrical circuits. We are interested in the NOT gate, the AND gate and the OR gate. We use diagrams to indicate the gates:



NOT



AND



OR

You should remember these symbols. It helps to imagine that the symbol for the AND gate is shaped like the D in AND that the symbol for the OR gate is curved on the left to fit the O of OR.

The AND gate and the OR gate have two input lines each and one output line. The NOT gate has one input line and one output line. It is possible for the AND and OR gates to have more than two input lines each – more about this later.

We construct **truth tables** to help us remember how each gate works:

| NOT | On  | Off |
|-----|-----|-----|
|     | Off | On  |

| AND | On  | Off |
|-----|-----|-----|
| On  | On  | Off |
| Off | Off | Off |

| OR  | On | Off |
|-----|----|-----|
| On  | On | On  |
| Off | On | Off |

These truth tables are drawn with “On” and “Off”. It is more usual to see truth tables showing TRUE (or T) for On and FALSE (or F) for Off. Sometimes we also write 1 for On and 0 for Off.

The truth tables provide us with the following knowledge about gates:

NOT gate      when the input line is on (or 1 or TRUE) then the output line is off  
                  when the input line is off (or 0 or FALSE) then the output line is on

AND gate      if both input lines are on then the output line is on (otherwise it is off)

OR gate        if both input lines are off then the output line is off (otherwise it is on)

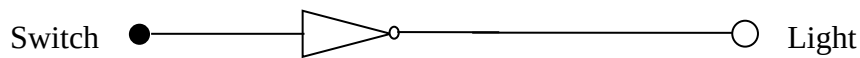
There are other gates. We will not cover all types of gates in our course. For more on gates:

[https://en.wikipedia.org/wiki/Logic\\_gate](https://en.wikipedia.org/wiki/Logic_gate)

## Logic circuits

We connect gates with lines (think of them as electric wires) to create circuits and we include switches and lights as needed. A switch can set a line to be on or off. A light glows if its input line is on.

Example: What does this circuit do:



When we flip the switch on, the light goes off and when we flip the switch off, the light comes on. This effect is caused by the NOT gate in the circuit.

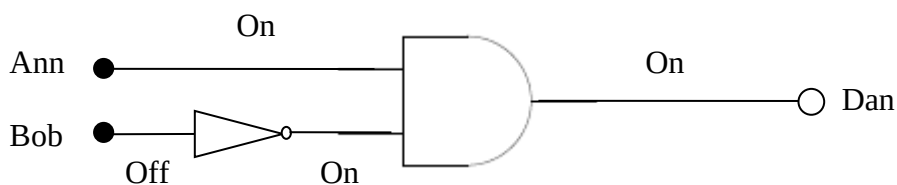
We now move on to a more interesting circuit to solve a specific problem, which is:

Ed will take 159.102 if Dan does or Carol does

Dan will take 159.102 if Ann does and Bob does not

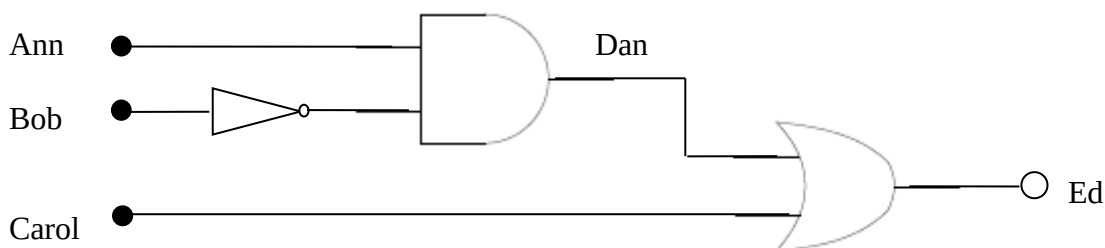
We will construct a logic circuit that will automatically tell Ed what to do. We use electricity to represent what each person does – if they take 159.102 we switch on, otherwise we switch off.

First we construct a circuit to see what Dan will do:



We have labelled the diagram for one particular situation. Work through every situation and label the diagram for each one of those situations.

Now we can build this circuit into a bigger circuit to see what Ed should do:



Check every possibility for this circuit.

For example, what happens if Ann, Bob and Carol all **do not take** 159.102?

This circuit performs a **logic function**. A function is a rule that takes in some input and produces some output. We are familiar with functions which take in numbers and produce another number. For example, if we provide the input “6” and “4” to the function “Add” we get the output “10”. Could we draw a truth table for “Add”? Why not?

A logic function takes in logic values (TRUE or FALSE) and produces another logic value. We can build complex logic functions (with many inputs) by combining simple logic functions.

## Boolean Algebra

Logic circuits take up a lot of space and time. We need a more concise notation. Algebra in mathematics uses variables to represent numbers. In Boolean Algebra we use variables to represent statements. The value of these statements can be TRUE or FALSE.

Example: Let  $R$  = “it is raining today” (this can be TRUE or FALSE)

### Symbols in Boolean Algebra

**We use  $\sim$  to represent not.** Thus  $\sim R$  means “it is not raining today”.

Assume  $N$  = “it is not raining today”. Then  $\sim R = N$ .

Note that  $\sim\sim R = R$ . This is true for any variable, i.e. it is true for any statement.

**We use  $+$  to represent OR.**

Thus  $R + S$  means: for the whole statement to be true –  $R$  must be true, or  $S$  must be true, or they must both be true.

(Aside: if we want  $R$  to be true or  $S$  to be true but not both to be true – we would use XOR).

**We use  $\bullet$  to represent AND.**

Thus  $R \bullet S$  means: for the whole statement to be true –  $R$  must be true and  $S$  must be true.

(Note that  $\bullet$  is often written simply as a full stop like this:  $R.S$ )

### Priority of Operations

Many of us learnt the rule BEDMAS to remind us of the order of priority of arithmetic operations.

The rule for Boolean Algebra is NOTANDOR (and also brackets).

Examples:

(Mathematics)  $4 + 2 * 3 = 10$  but  $(4 + 2) * 3 = 18$

(Boolean Algebra)  $H + J \bullet K = \text{TRUE}$  but  $(H + J) \bullet K = \text{FALSE}$   
(assume  $H = \text{TRUE}$ ,  $J = \text{FALSE}$ ,  $K = \text{FALSE}$ )

### Evaluating Expressions

In Mathematics we could have the expression  $x + y$ . We cannot evaluate the expression until we assign values to  $x$  and  $y$ . For example, if  $x = 2$  and  $y = 3$  then  $x + y = 5$ .

Similarly in Boolean Algebra we can evaluate expressions once we assign values to variables.

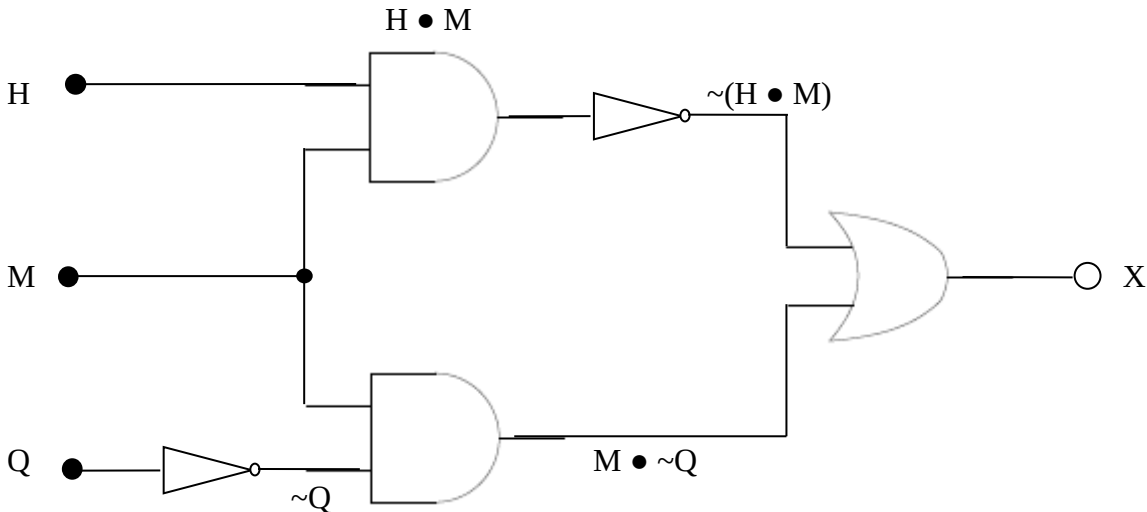
Assume  $P = \text{FALSE}$  and  $Q = \text{TRUE}$  and evaluate:  $(P + \sim P) \bullet \sim\sim Q$

$(P + \sim P) \bullet \sim\sim Q = (P + \sim P) \bullet Q = (\text{FALSE} + \text{TRUE}) \bullet \text{TRUE} = \text{TRUE} \bullet \text{TRUE} = \text{TRUE}$

Note that for any statement  $P + \sim P$  is always true.

You need to be able to switch between Boolean Algebra, logic circuits and truth tables.

For example:



Boolean Expression:  $X = \sim(H \bullet M) + (M \bullet \sim Q)$

X is known as the output of the circuit. We usually put brackets around each **term**.

The corresponding truth table is:

| H | M | Q | Output |
|---|---|---|--------|
| F | F | F | T      |
| F | F | T | T      |
| F | T | F | T      |
| F | T | T | T      |
| T | F | F | T      |
| T | F | T | T      |
| T | T | F | T      |
| T | T | T | F      |

Some things to notice about the truth table:

We set up a pattern to draw the truth table – Q changes F,T,F,T,etc, M changes F,F,T,T,F,F,etc, and H changes in fours. By using this pattern we ensure that we cover all possible situations.

We notice patterns in the output, e.g. if H is FALSE then the output is true. Also if M is FALSE then the output is true. This enables us to take some short cuts in the design of the circuit.

If designing the circuit from the truth table, make sure that you include all situations where the output is TRUE. We can ignore any situations where the output is FALSE.

### Summary

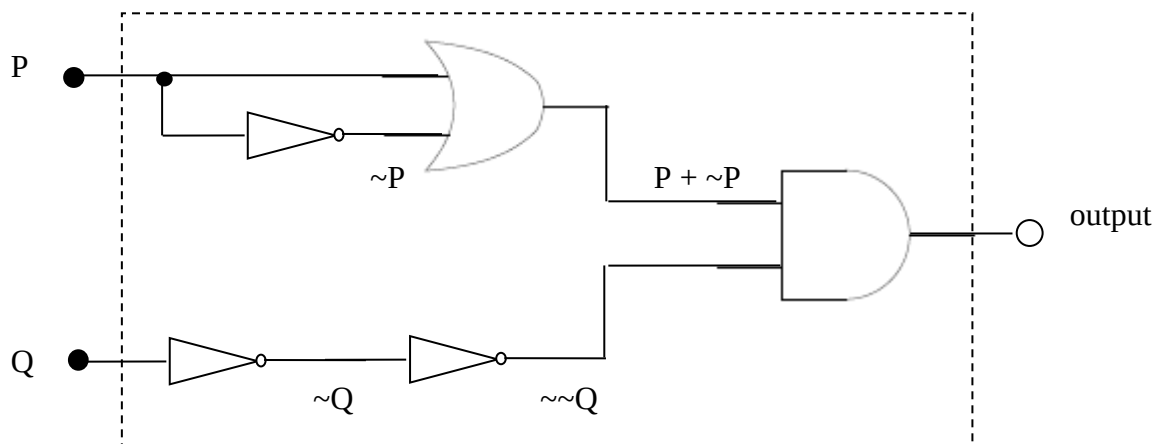
We study logic circuits because they are the basis for designing a computer.

We study Boolean Algebra as a short-hand notation for writing logic circuits. The Boolean Algebra also assists us to find ways of reducing the circuit by removing components, e.g.  $\sim\sim R = R$ .

We use truth tables to check that our circuit is functioning for every situation.

## Reducing Boolean Expressions

Let us set up the logic circuit for  $(P + \sim P) \bullet \sim\sim Q$  (We make a “black box” with 2 inputs and 1 output).



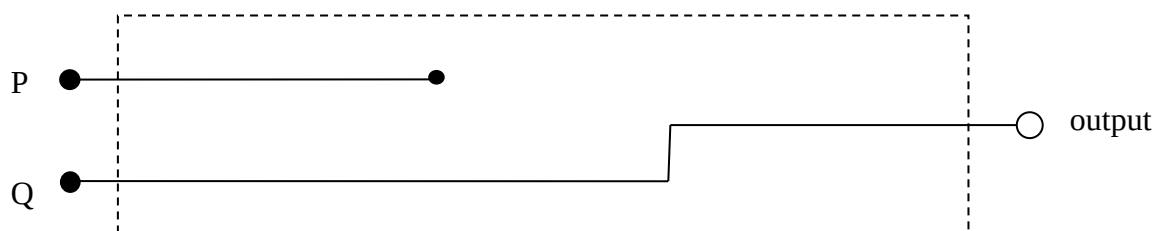
We can now **reduce this circuit** by using Boolean Algebra as follows:

Reduce  $(P + \sim P) \bullet \sim\sim Q$

Step 1:  $\sim\sim Q = Q$  so we reduce to  $(P + \sim P) \bullet Q$

Step 2:  $P + \sim P = \text{true}$  so we reduce to  $\text{true} \bullet Q$

Step 3:  $\text{true} \bullet Q = Q$  so we reduce to  $Q$



Note that the two “black boxes” do the same thing – you can prove it with a truth table!

By using Boolean Algebra we can reduce Boolean expressions. Reduced Boolean expressions enable us to build smaller circuits in computers. Smaller circuits save **space** (computers get smaller with bigger memory) and **money** (less components so cheaper to build).

## Axioms

Axioms are the **laws** or **rules** that form the foundation of any algebra.

### The Axioms of Boolean Algebra

|                                                            |                                                                         |              |
|------------------------------------------------------------|-------------------------------------------------------------------------|--------------|
| $P + Q = Q + P$                                            | $P \bullet Q = Q \bullet P$                                             | Commutative  |
| $(P + Q) + R = P + (Q + R)$                                | $(P \bullet Q) \bullet R = P \bullet (Q \bullet R)$                     | Associative  |
| $P \bullet Q + P \bullet R = P \bullet (Q + R)$            | $(P + Q) \bullet (P + R) = P + (Q \bullet R)$                           | Distributive |
| $\text{TRUE} + P = \text{TRUE} \quad \text{FALSE} + P = P$ | $\text{TRUE} \bullet P = P \quad \text{FALSE} \bullet P = \text{FALSE}$ | Identities   |

$$P + \sim P = \text{TRUE}$$

$$P \bullet \sim P = \text{FALSE}$$

Complement

$$P + P = P$$

$$P \bullet P = P$$

Idempotent

$$\sim \sim P = P$$

Involution

Because Boolean algebra only has two values (TRUE and FALSE) we can use truth tables to show that an axiom is true for every possibility.

For example:

Use a truth table to show that  $P \bullet Q + P \bullet R = P \bullet (Q + R)$  is always TRUE:

| P | Q | R | Q + R | P • (Q + R) | P • Q | P • R | P•Q + P•R |
|---|---|---|-------|-------------|-------|-------|-----------|
| F | F | F | F     | F           | F     | F     | F         |
| F | F | T | T     | F           | F     | F     | F         |
| F | T | F | T     | F           | F     | F     | F         |
| F | T | T | T     | F           | F     | F     | F         |
| T | F | F | F     | F           | F     | F     | F         |
| T | F | T | T     | T           | F     | T     | T         |
| T | T | F | T     | T           | T     | F     | T         |
| T | T | T | T     | T           | T     | T     | T         |

RHS

LHS

The truth table shows that the LHS = RHS for every possibility.

## How to design a circuit

Assume we have a problem that requires a circuit – understand the problem.

Create a truth table to cover every possibility that will exist for the new circuit.

Convert the truth table into a Boolean expression.

Reduce the expression using the axioms of Boolean algebra.

Design a circuit that corresponds to the reduced Boolean expression.

Check that the circuit matches the truth table for every situation.

Example: the 3-way voting circuit

**A. The Problem:** there are three people on a committee. They are called A, B and C. When they vote on a proposal, the proposal will be approved if there are 2 or 3 votes in favour of the proposal and the proposal will fail if there are 0 or 1 votes for it. Design a voting circuit in which there is a switch for each person and a light that comes on if the proposal has enough votes.

**B. Truth Table:** create a truth table to cover every possible situation

| A | B | C | Output |
|---|---|---|--------|
| 0 | 0 | 0 | 0      |
| 0 | 0 | 1 | 0      |
| 0 | 1 | 0 | 0      |

|   |   |   |   |
|---|---|---|---|
| 0 | 1 | 1 | 1 |
| 1 | 0 | 0 | 0 |
| 1 | 0 | 1 | 1 |
| 1 | 1 | 0 | 1 |
| 1 | 1 | 1 | 1 |

### C. Convert the truth table to Boolean expression:

Ignore any rows with an output of F. Each row becomes a **term**. Link the terms together with OR.  
So we get:  $A \bullet B \bullet C + A \bullet B \bullet \sim C + A \bullet \sim B \bullet C + \sim A \bullet B \bullet C$

**D. Reduce the Expression:** we start by *increasing* the number of terms using the axiom  $P = P + P$ .  
We also make use of the axiom that  $P + Q = Q + P$ , so we get:

$$A \bullet B \bullet C + A \bullet B \bullet \sim C + A \bullet B \bullet C + A \bullet \sim B \bullet C + A \bullet B \bullet C + \sim A \bullet B \bullet C$$

Now we use the axiom  $P \bullet Q + P \bullet R = P \bullet (Q + R)$  to get:

$$A \bullet B \bullet (C + \sim C) + A \bullet C \bullet (B + \sim B) + B \bullet C \bullet (A + \sim A)$$

Now we use the axiom  $P + \sim P = \text{true}$  and the axiom  $P \bullet \text{true} = P$  to get:

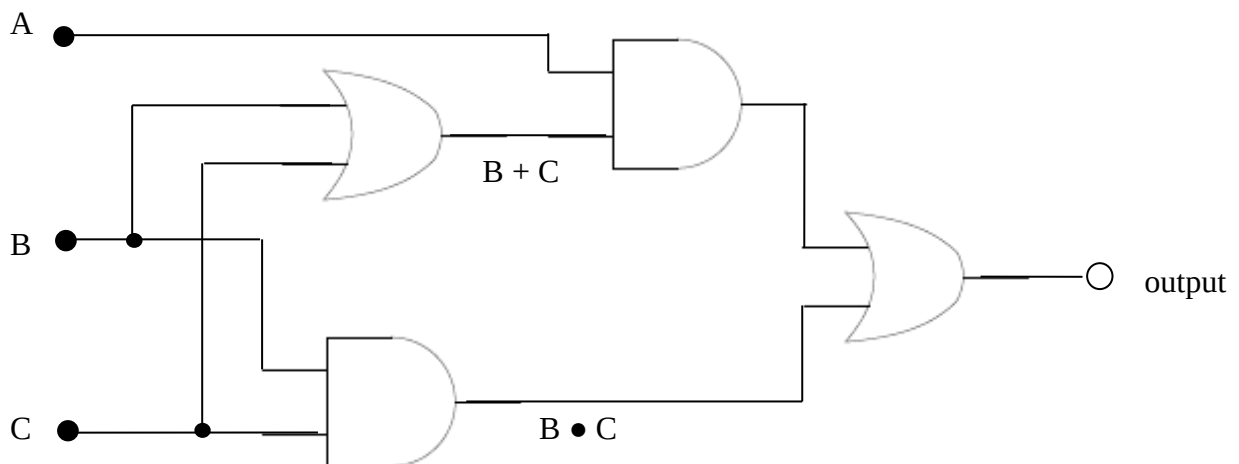
$$A \bullet B + A \bullet C + B \bullet C$$

Finally we again use the axiom  $P \bullet Q + P \bullet R = P \bullet (Q + R)$  to get:

$$A \bullet (B + C) + B \bullet C$$

**IMPORTANT:** check the reduced expression against every situation in the truth table!!

### E. Design the circuit:



**IMPORTANT:** check that the circuit does what it is meant to do!!

## De Morgan's Laws

The English mathematician Augustus De Morgan (in about 1850) established two further laws:

$$\sim(P \bullet Q) = \sim P + \sim Q \quad \text{and} \quad \sim(P + Q) = \sim P \bullet \sim Q$$

We can show that they are correct for every situation by looking at the truth tables, for example:

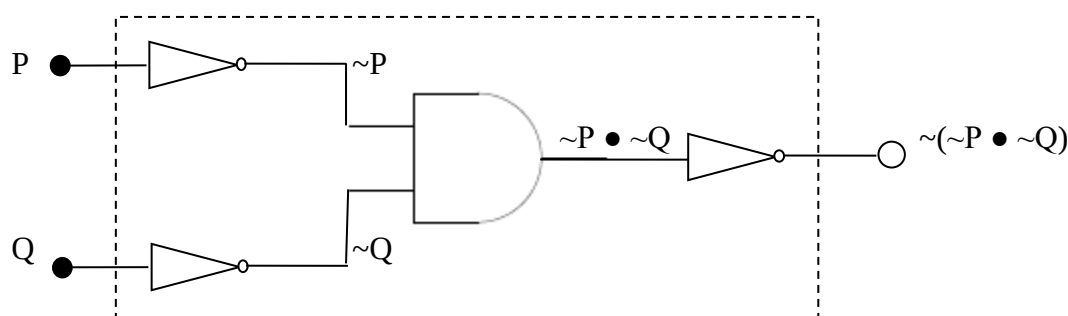
| P | Q | $P \bullet Q$ | $\sim(P \bullet Q)$ | $\sim P$ | $\sim Q$ | $\sim P + \sim Q$ |
|---|---|---------------|---------------------|----------|----------|-------------------|
| F | F | F             | T                   | T        | T        | T                 |
| F | T | F             | T                   | T        | F        | T                 |
| T | F | F             | T                   | F        | T        | T                 |
| T | T | T             | F                   | F        | F        | F                 |

LHS

RHS

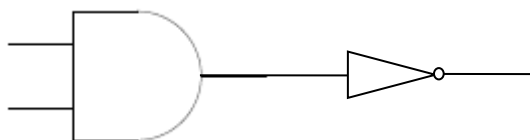
De Morgan's Laws are important as they allow us to convert AND into OR or vice versa.

Let us study the following circuit:

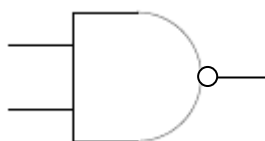


The output is  $\sim(\sim P \bullet \sim Q) = \sim(\sim(P + Q)) = P + Q$ . **Thus this circuit does the same thing as an OR gate but it does not include an OR gate.** De Morgan's Laws enable us to build an OR gate by using AND and NOT.

One of the components inside this circuit was:



This combination of AND and NOT occurs frequently so we create a new gate by combining them.

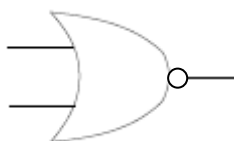


| NAND | T | F |
|------|---|---|
| T    | F | T |
| F    | T | T |

This is called a **NAND gate**.

The truth table for the NAND gate.

Similarly, we can develop the **NOR gate** with symbol and truth table as follows:

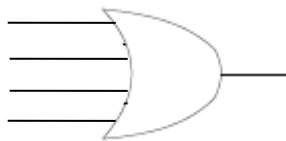


| NOR | T | F |
|-----|---|---|
| T   | F | F |
| F   | F | T |

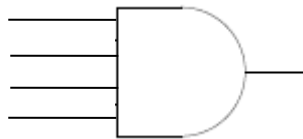
All other gates can be constructed by using NAND gates and NOT gates.

### Gates with multiple inputs:

$A + B + C + D$  can be drawn as:



$A \bullet B \bullet C \bullet D$  can be drawn as:



Two little rules that can be helpful when you are looking at gates with multiple inputs:

OR gate: you need just one input line to be true to make the output true.  
the output is false only if all input lines are false.

AND gate: you need just one input line to be false to make the output false  
the output is true only if all input lines are true.

### **Working through a typical problem**

Here is a truth table. Write down the corresponding Boolean expression, reduce the expression and design the circuit. Check the final circuit against the truth table.

Note: you need to be able to answer this type of question from any point in the cycle:

Truth table  $\rightarrow$  Boolean expression  $\rightarrow$  circuit  $\rightarrow$  truth table  $\rightarrow$  ....

Example: start with the following truth table:

| A | B | C | Output |
|---|---|---|--------|
| F | F | F | T      |
| F | F | T | T      |
| F | T | F | F      |
| F | T | T | F      |
| T | F | F | F      |
| T | F | T | F      |
| T | T | F | F      |
| T | T | T | T      |

To construct the Boolean expression, ignore all rows ending with F and so we get:

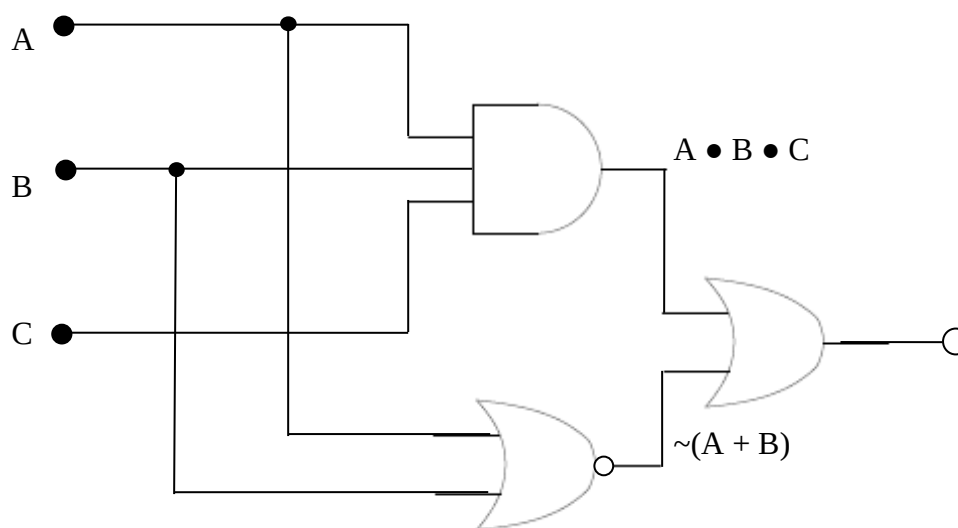
$$A \bullet B \bullet C + \sim A \bullet \sim B \bullet C + \sim A \bullet \sim B \bullet \sim C$$

Using the axiom  $P \bullet Q + P \bullet R = P \bullet (Q + R)$  we get:  $A \bullet B \bullet C + \sim A \bullet \sim B \bullet (C + \sim C)$

Using  $P + \sim P = \text{true}$  and  $\text{true} \bullet P = P$  we get:  $A \bullet B \bullet C + \sim A \bullet \sim B$

And using De Morgan's Law we get:  $A \bullet B \bullet C + \sim(A + B)$

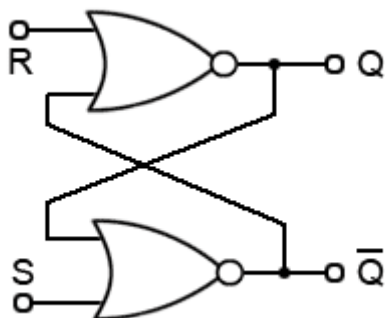
And now we can design the circuit...



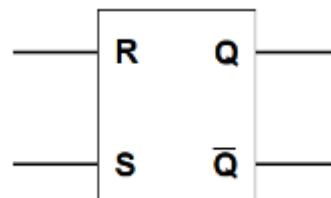
Now go back to the truth table and check the circuit against each row of the table. For example:  
if A is OFF (FALSE) and B and C are both ON (TRUE) then the output should be OFF (FALSE).

## The R-S flip-flop (also called a latch)

The circuit for the R-S flip-flop (where S means Set and R means Reset):



The “black box” diagram for the R-S flip-flop:



This circuit is rather difficult to follow but we will just accept it and not try to figure out exactly how it works. Looking at the black box (simplified) version of the R-S flip-flop we can see that it has two input lines (R and S) and two output lines (Q and  $\sim Q$ ). When S is switched briefly on (and then off again), Q will be set to on (or TRUE or 1). When R is switched briefly on (and then off again), Q will be set to off (or FALSE or 0). Q is called the **state** of the flip-flop. This is a **bistable circuit**, i.e. it has two states (on and off). The flip-flop can be used as a memory cell capable of storing zero (off) or one (on). We want to store larger numbers so we group a number of cells together to form a **memory location**.

|          |          |          |          |          |          |          |          |
|----------|----------|----------|----------|----------|----------|----------|----------|
| flipflop | flipflop | flipflop | flipflop | flipflop | flipflop | flipflop | flipflop |
| ON       | OFF      | OFF      | ON       | OFF      | ON       | ON       | OFF      |
| 1        | 0        | 0        | 1        | 0        | 1        | 1        | 0        |

Thus a **memory location** is a group of flipflops (circuits) that work together to store a **binary number** represented by zeros and ones.

## **Logic circuit simulators – for interest only**

It is a good idea to use a logic circuit simulator to assist you with drawing your circuits. There are a number of applications online including:

<https://logic.ly/demo>

This is my favourite one (and has the adder circuit built in) but unfortunately it costs money.

<https://academo.org/demos/logic-gate-simulator/>

This is a free simulator and does everything we need.

There are definitely other logic circuit simulators out there. To find more just do a google search for “logic circuit simulator”.

WARNING – do not search for “circuit simulator” because that takes you deep into the realm of REAL circuit design including voltages, etc. Fascinating stuff but much deeper than we want to go.